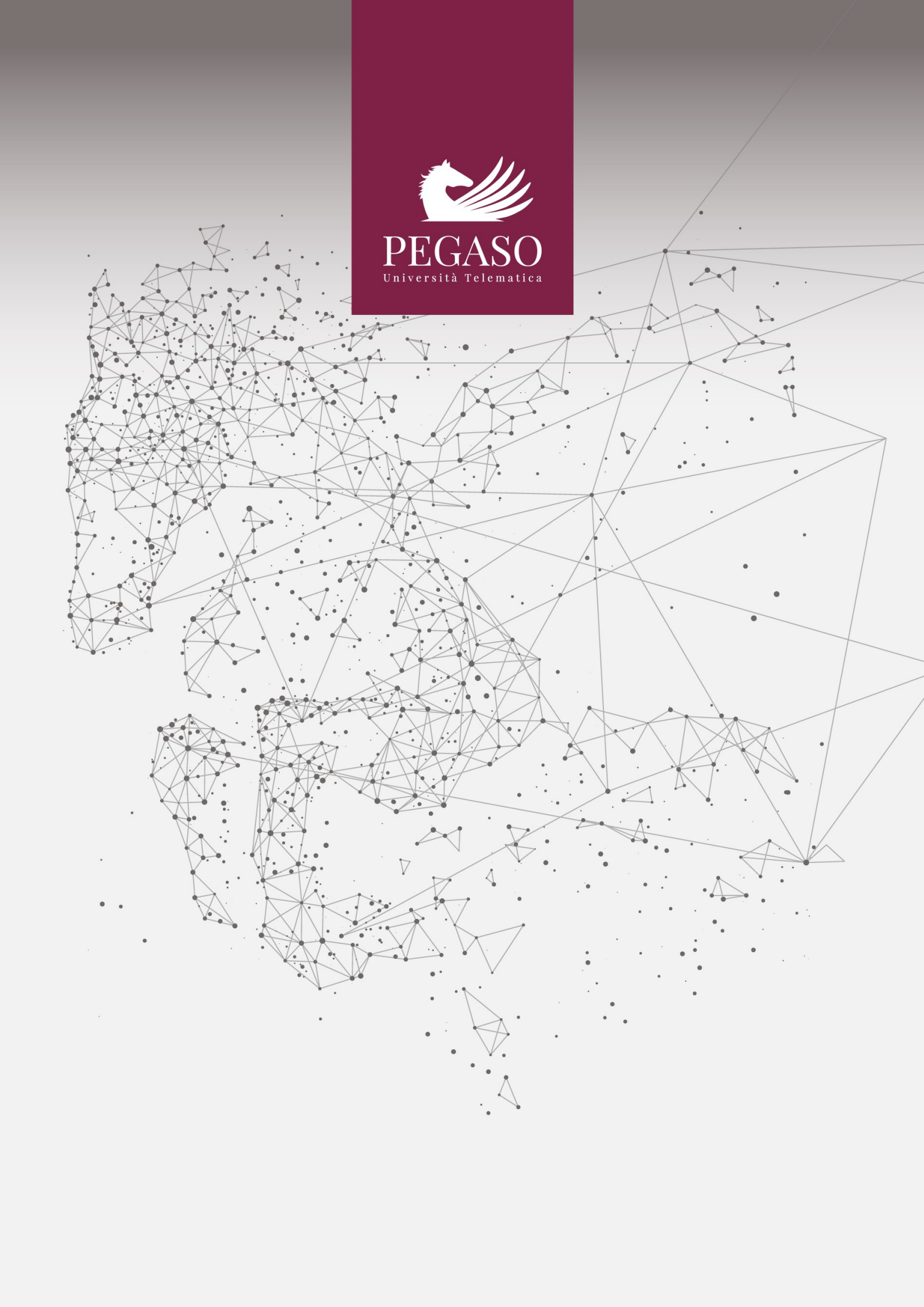

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. MVC 3
2. MODULE 5
3. VIEWS 7
BIBLIOGRAFIA 15
SITOGRAFIA 16

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. MVC

MVC (Model-View-Controller) è un pattern architetturale utilizzato per separare le responsabilità del codice in tre parti distinte: il **Model**, la **View** e il **Controller**.

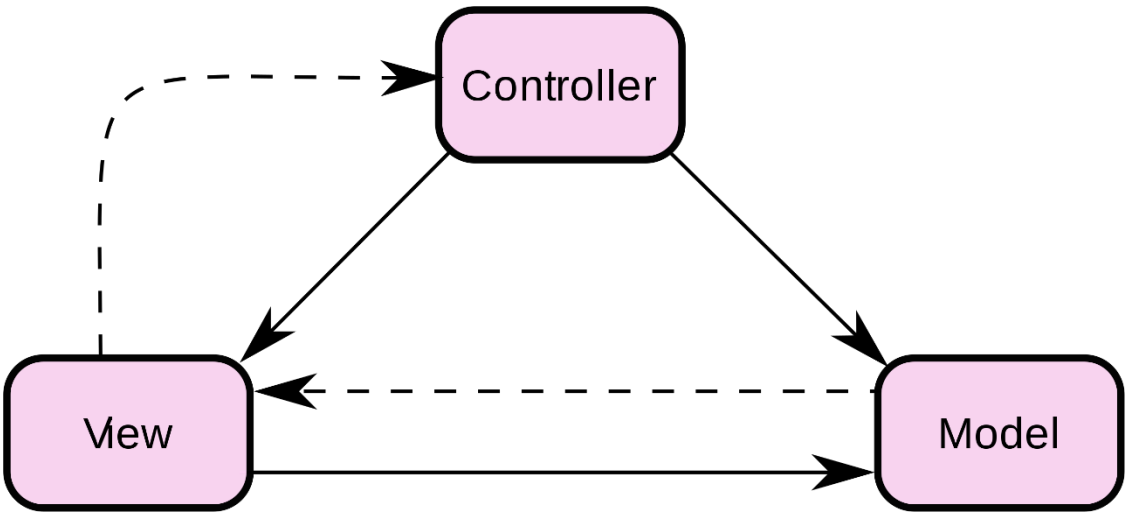


***Figura 1** – MVC*

- Il **Model** rappresenta i dati dell'applicazione e definisce le operazioni per accedere e manipolare questi dati. Il Model non conosce nulla della View o del Controller, ma fornisce solo un'interfaccia per accedere ai dati.
- La **View** rappresenta l'interfaccia utente dell'applicazione e visualizza i dati forniti dal Model. La View non ha alcuna conoscenza del Model o del Controller, ma fornisce solo un'interfaccia utente per interagire con l'applicazione.
- Il **Controller** si occupa di coordinare l'interazione tra il Model e la View. Il Controller riceve le richieste dell'utente dalla View, accede ai dati del Model per elaborare la richiesta e aggiorna la View con i dati elaborati.

L'utilizzo del pattern MVC ha diversi vantaggi, tra cui una maggiore separazione delle responsabilità del codice, una maggiore modularità, una maggiore flessibilità e una maggiore facilità di manutenzione e testabilità.

In Node.js, il pattern MVC può essere implementato utilizzando un framework web come Express.js. Ad esempio, si può utilizzare Express per gestire le richieste dell'utente (Controller), definire rotte e middleware per l'accesso ai dati dell'applicazione (Model) e utilizzare template engine come Handlebars o EJS per creare le View.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Vediamo nel dettaglio i singoli componenti del pattern MVC e come possono essere implementati in Node.js.

- Il **Model** rappresenta i dati dell'applicazione e definisce le operazioni per accedere e manipolare questi dati. In Node.js, il Model può essere implementato utilizzando una libreria di accesso ai database come Sequelize, Mongoose o Knex.js. Ad esempio, Sequelize è un ORM (Object-Relational Mapping) che permette di definire i modelli dei dati dell'applicazione e di interagire con il database utilizzando oggetti JavaScript.
- La **View** rappresenta l'interfaccia utente dell'applicazione e visualizza i dati forniti dal Model. In Node.js, la View può essere implementata utilizzando un template engine come Handlebars, EJS o Pug. Ad esempio, Handlebars permette di creare template HTML dinamici utilizzando una sintassi simile a quella di Mustache.
- Il **Controller** si occupa di coordinare l'interazione tra il Model e la View. In Node.js, il Controller può essere implementato utilizzando le funzionalità di routing di Express.js. Ad esempio, si può utilizzare Express per definire le rotte dell'applicazione e le funzioni che gestiscono le richieste HTTP. In questo modo, il Controller accede ai dati del Model per elaborare la richiesta e aggiorna la View con i dati elaborati.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Module

Un modulo è un file che contiene codice, variabili, funzioni o oggetti che possono essere esportati e utilizzati in altri moduli.

La variabile module.exports è una proprietà dell'oggetto module che viene utilizzata per esportare funzioni, oggetti, variabili o qualsiasi altra cosa presente in un modulo. Ad esempio, se vogliamo esportare una funzione dal nostro modulo, possiamo farlo in questo modo:

```
// Definiamo una funzione all'interno del nostro modulo
function myFunction() {
  console.log('Hello, world!');
}

// Esportiamo la funzione tramite module.exports
module.exports = myFunction;
```

In questo esempio, abbiamo definito una funzione myFunction() all'interno del nostro modulo e l'abbiamo esportata utilizzando la variabile module.exports. Questo rende la funzione disponibile per l'importazione in altri moduli.

Per importare la funzione nel nostro codice, possiamo utilizzare la funzione require(). Ad esempio:

```
// Importiamo la funzione dal nostro modulo
const myFunction = require('./myModule');

// Utilizziamo la funzione
myFunction(); // Output: "Hello, world!"
```

In questo esempio, abbiamo importato la funzione myFunction() dal nostro modulo utilizzando la funzione require(). Abbiamo quindi chiamato la funzione utilizzando il nome della variabile a cui è stata assegnata l'importazione.

Generalizzando, lvi sono tre principali modalità per esportare un modulo in Node.js.

Esportare singole funzioni: questo è utile se si ha solo una funzione nel modulo che si vuole utilizzare in un altro modulo; si può esportare una funzione alla volta utilizzando module.exports

```
function myFunction() {
    console.log('Hello World');
}
function anotherFunction() {
    console.log('Hello World2');
}
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
module.exports = myFunction,anotherFunction;
```

Esportare un oggetto con i metodi: questo è utile se si hanno diverse funzioni correlate che possono essere raggruppate in un oggetto, utilizzando il modulo come un "namespace" per i metodi; si può creare un oggetto contenente i metodi e utilizzarlo come oggetto esportato

```
const myModule = {
    method1() { console.log('method 1'); },
    method2() { console.log('method 2'); },
    method3() { myModule.method2();console.log('method 3'); }
};

module.exports = myModule;
```

Da notare come qualora si volesse usare un metodo all'interno di un altro, occorra esplicitamente usare il nome del modulo.

Esportare una classe: questo è utile se si deve mantenere lo stato all'interno del modulo. Si può esportare una classe e creare oggetti istanza della classe. Ad esempio:

```
class MyClass {
    constructor() {
        this.myProperty = 'Hello World';
    }

    myMethod() {
        console.log(this.myProperty);
    }
}

module.exports = MyClass;
```

Ovviamente questi tre metodi di esportazione non sono esclusivi l'uno dall'altro e possono essere combinati all'interno di uno stesso modulo, in base alle esigenze specifiche.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Views

Le view sono la parte dell'applicazione che definisce come i dati dell'applicazione devono essere presentati all'utente. In Express, le view possono essere implementate utilizzando uno dei molti motori di template disponibili, come ad esempio EJS (Embedded JavaScript).

EJS è un motore di template che permette di incorporare codice JavaScript all'interno di pagine HTML. Utilizzando EJS, è possibile definire modelli di pagina che vengono compilati dinamicamente con i dati dell'applicazione.

Per utilizzare EJS in un progetto Express, bisogna innanzitutto installarlo tramite npm: npm install ejs

Una volta installato EJS, si può configurare l'applicazione per utilizzare EJS come motore di template:

```
const express = require('express');
const app = express();
app.set('view engine', 'ejs');
```

In questo esempio, si utilizza il metodo app.set() per impostare il motore di template predefinito dell'applicazione su EJS.

Per creare una view con EJS, si deve creare un file con estensione .ejs che definisce la struttura della pagina. Questo file può contenere codice HTML, CSS e JavaScript, oltre a tag EJS che consentono di incorporare i dati dell'applicazione.

Ad esempio, si potrebbe creare una view hello.ejs che mostra un messaggio di saluto:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Saluto</title>
  </head>
  <body>
    <h1>Ciao, <%= name %>!</h1>
  </body>
</html>
```

In questo esempio, il tag EJS <%= name %> viene utilizzato per visualizzare il valore della variabile name, che sarà passata alla view come parametro.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Per rendere la view disponibile all'applicazione, si deve definire una rotta che renderizza la view. Ad esempio:

```
app.get('/hello/:name', (req, res) => {
  const name = req.params.name;
  res.render('hello', { name });
});
```

In questo esempio, si definisce una rotta per la richiesta HTTP GET all'URL /hello/:name, dove :name rappresenta un parametro dinamico. Quando viene ricevuta una richiesta su questa rotta, viene estratto il valore del parametro dinamico name dalla richiesta utilizzando req.params.name, e viene passato alla view utilizzando il metodo res.render().

Il metodo res.render() viene utilizzato per renderizzare la view e inviare la pagina HTML compilata al client. Il primo parametro del metodo res.render() indica il nome del file .ejs che contiene la view. Il secondo parametro è un oggetto che contiene i dati che devono essere passati alla view. In questo esempio, l'oggetto { name } viene passato alla view con la variabile name impostata al valore del parametro dinamico name estratto dalla richiesta.

In questo modo, la view hello.ejs viene compilata dinamicamente con i dati passati come parametro e inviata al client come risposta HTTP.

Facciamo ora un altro esempio:

```
const express = require('express');
const app = express();
const port = 3000;

app.set('view engine', 'ejs');

app.get('/', (req, res) => {
  const title = 'Homepage';
  const items = [
    { id: 1, name: 'Item 1' },
    { id: 2, name: 'Item 2' },
    { id: 3, name: 'Item 3' },
  ];
  res.render('index', { title, items });
});

app.get('/item/:id', (req, res) => {
  const id = req.params.id;
  const item = { id, name: `Item ${id}` };
  res.render('item', { item });
});
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
app.listen(port, () => {
  console.log(`App listening at http://localhost:${port}`);
});
```

In questo esempio, abbiamo definito due rotte GET:

- una per la homepage;
- una per visualizzare un singolo item.

Nella rotta della homepage, abbiamo creato due variabili: title e items. La variabile title contiene il titolo della pagina, mentre la variabile items è un array di oggetti che rappresentano gli item da visualizzare sulla pagina. Abbiamo poi utilizzato il metodo res.render() per renderizzare la view index.ejs, passando come argomento un oggetto con le variabili da utilizzare nella view.

Nella rotta dell'item, abbiamo creato una variabile item che rappresenta l'item da visualizzare. Abbiamo poi utilizzato il metodo res.render() per renderizzare la view item.ejs, passando come argomento un oggetto con la variabile item.

Entrambe le view index.ejs e item.ejs utilizzano la sintassi EJS per accedere alle variabili passate attraverso il metodo res.render(). Ad esempio, nella view index.ejs, utilizziamo il seguente codice per ciclare attraverso gli item e visualizzarli sulla pagina:

```
<h1><%= title %></h1>
<ul>
  <% items.forEach((item) => { %>
    <li><%= item.name %></li>
  <% }) %>
</ul>
```

In questo esempio, utilizziamo la sintassi EJS <% %> per eseguire il codice JavaScript, e la sintassi <%= %> per visualizzare il valore di una variabile all'interno della pagina.

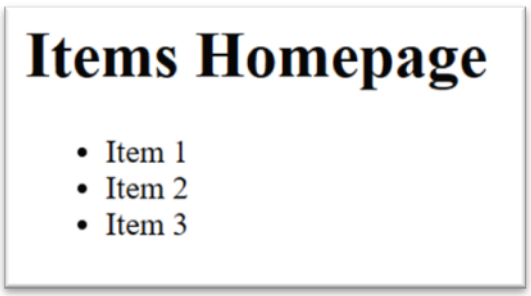


Figura 2 *- Items*

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La view item.ejs utilizza un approccio simile per visualizzare le informazioni sull'item:

```
<h1>Item <%= item.id %></h1>
<p><%= item.name %></p>
```

In questo esempio, utilizziamo la variabile item passata alla view per visualizzare l'ID e il nome dell'item sulla pagina.

Ecco un esempio di chiamata:

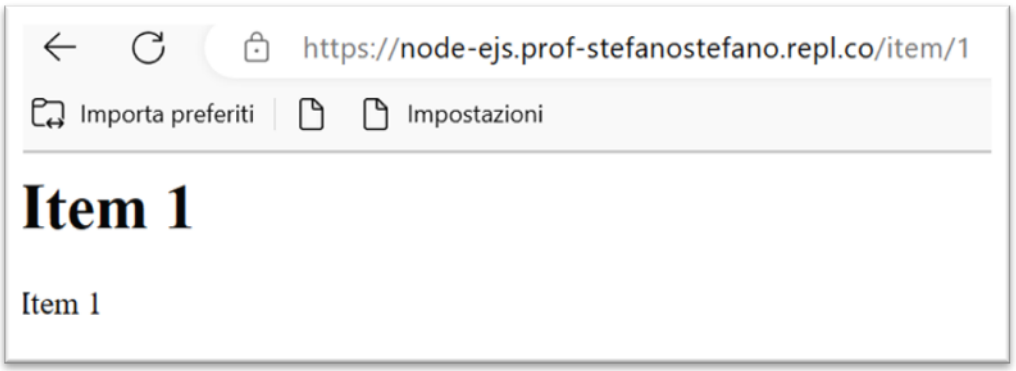


Figura 3 *- Item*

Costruiamo ora una nuova route:

```
const myformRouter = require('./routes/myform');
app.use('/myform', myformRouter);
```

A questo punto occorre inserire un nuovo file myform.js all'interno di routes/

```
const express = require('express');
const bodyParser = require('body-parser');
const router = express.Router();

router.use(bodyParser.urlencoded({ extended: false }));

router.get('/', (req, res) => {
  res.render('form');
});

router.post('/', (req, res) => {
  const name = req.body.name;
  const email = req.body.email;
  const message = req.body.message;

  res.render('success', { name, email, message });
});

module.exports = router;
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Inseriamo le nostre views:

form.ejs

```
<!DOCTYPE html>
<html>
  <head>
    <title>Form</title>
  </head>
  <body>
    <h1>Form</h1>
    <form action="/myform" method="POST">
      <label for="name">Name:</label>
      <input type="text" name="name" required>
      <br>
      <label for="email">Email:</label>
      <input type="email" name="email" required>
      <br>
      <label for="message">Message:</label>
      <textarea name="message" required></textarea>
      <br>
      <input type="submit" value="Submit">
    </form>
  </body>
</html>
```

success.ejs:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Success</title>
  </head>
  <body>
    <h1>Success</h1>
    <p>Thank you for your submission, <%= name %>!</p>
    <p>We will contact you at <%= email %> soon.</p>
    <p>Your message was: <%= message %></p>
  </body>
</html>
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

In questo caso richiamando la url con la form otteniamo:

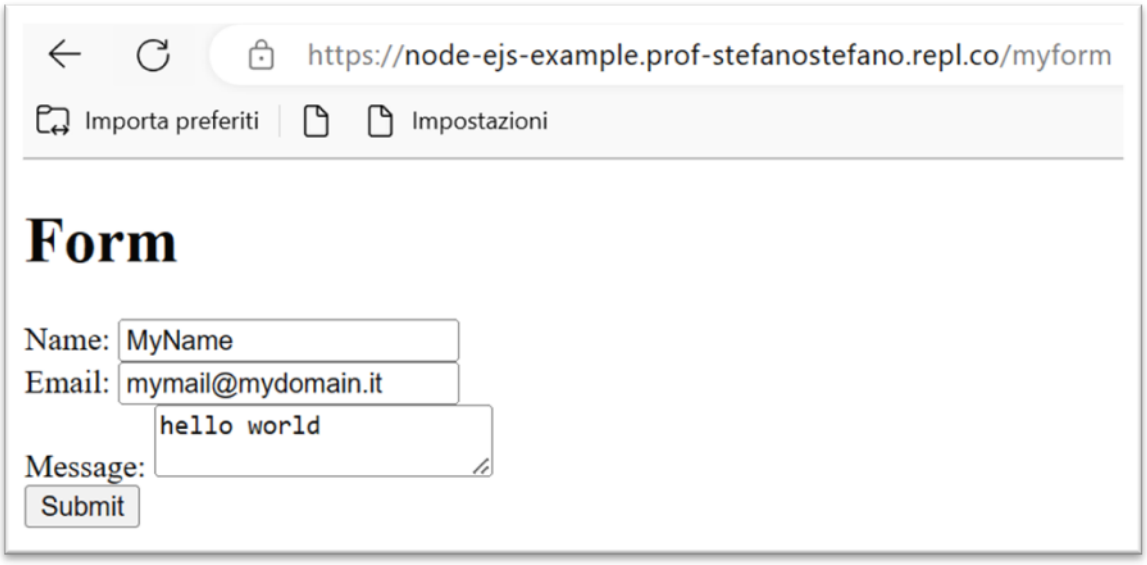


Figura 4 *- form*

Eseguendo il submit si ottiene:

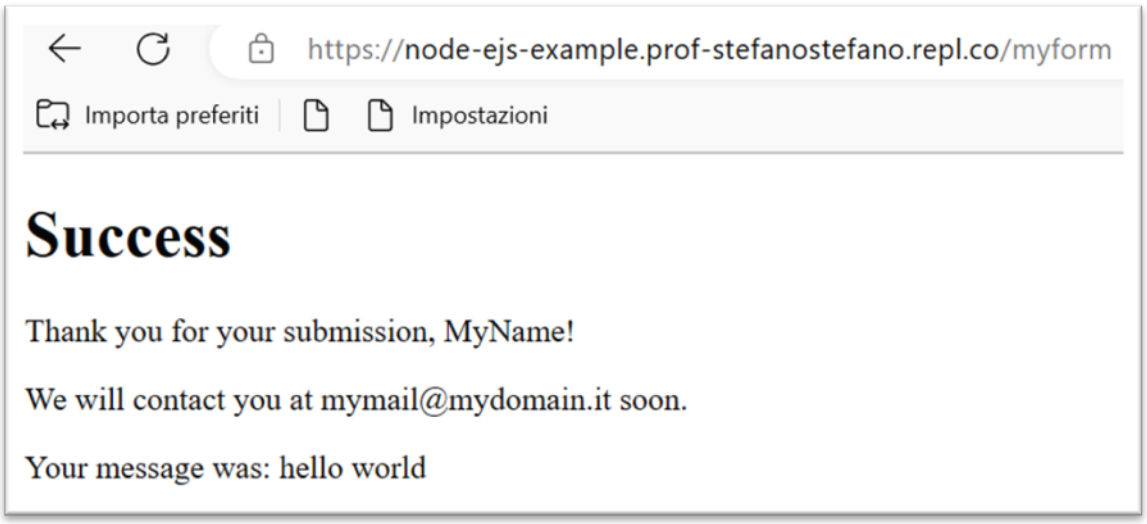


Figura 5 *- success*

Riportiamo di seguito alcune funzionalità da poter usare in EJS.

È possibile utilizzare la sintassi for o forEach per iterare un array o un oggetto e generare HTML dinamicamente. Ad esempio:

```
<ul>
  <% for (let i = 0; i < users.length; i++) { %>
    <li><%= users[i].name %></li>
  <% } %>
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
</ul>

<ul>
  <% users.forEach(user => { %>
    <li><%= user.name %></li>
  <% }) %>
</ul>
```

È possibile utilizzare la sintassi if e else per creare blocchi di codice HTML condizionati. Ad esempio:

```
<% if (user.isAdmin) { %>
  <p>Welcome, admin!</p>
<% } else { %>
  <p>Welcome, user!</p>
<% } %>
```

È possibile creare file parziali con parti di codice HTML comune a più view e includerli utilizzando la sintassi <%- include('partial') %>:

```
<html>
  <head>
    <title><%= title %></title>
    <%- include('partials/head') %>
  </head>
  <body>
    <%- include('partials/header') %>

    <h1><%= title %></h1>
    <p><%= content %></p>

    <%- include('partials/footer') %>
  </body>
</html>
```

È possibile utilizzare variabili globali nel template EJS, ad esempio per passare informazioni sullo stato dell'applicazione. Per fare ciò, utilizzare la sintassi <%= locals.variableName %>; nel nostro file javascript possiamo definire ad esempio:

```
app.locals.appName = 'My App';
```

Nel file .ejs possiamo accedere alla variabile nella seguente modalità:

```
<h1><%= locals.appName %></h1>
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Ci sono anche altre funzionalità avanzate che potrebbero essere utili a seconda delle esigenze dell'applicazione, come ad esempio la sintassi include con parametri, l'uso di layout, la definizione di funzioni custom, ecc

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- V. Roberto, M. Frailis, A. Gugliotta, P. Omero. Introduzione alle Tecnologie Web, McGraw-Hill, 2010.
- A. S. Tanembaum. Computer Networks (4 ed.). Prentice Hall (4 ed.), 2003.
- Marco Andreini, Patrizia Andronico, Arianna Del Soldato, Cristian Lucchesi, Maurizio Martinelli, Giuseppe Vasarelli. Manuale corso di Tecnologie Web Istituto di Informatica e Telematica del CNR, 2005.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Sitografia

- HTML 4.01 Specification: http://www.w3.org/TR/html401/
- HTML5: http://www.w3.org/TR/html5/
- Javascript: JavaScript Tutorial (w3schools.com)
- HTML: http://www.w3schools.com/html/default.asp
- CSS: https://www.w3schools.com/css/default.asp
- CSS: http://www.w3.org/TR/CSS/
- NODEJS: https://nodejs.org

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).